

Sentinel AI

AI-Assisted Security Operations Center (SOC) Analyst System

IBM HACKATHON — 5-DAY PROTOTYPE

1. PROJECT OVERVIEW

Sentinel AI is a backend-first security operations prototype that connects a web application's activity logs to an AI-assisted investigation and code-audit pipeline. The problem it addresses is the manual workload placed on SOC analysts: correlating high-volume HTTP traffic, identifying patterns that indicate attack attempts, then locating and explaining the vulnerable code responsible — a process that normally spans multiple tools and requires significant analyst time.

The system is designed to help a security analyst working against a web application they are authorized to audit. It does not make autonomous decisions or modify any production system; it surfaces findings and evidence for the analyst to review.

2. CORE WORKFLOW

① Web Application	A monitored web application (the "Meridian Analytics" demo target: Flask + SQLite, running locally on port 5001) emits HTTP access logs to the backend via POST /logs.
② Log Ingestion	The LogProcessor ingests each log, stores it in memory thread-safely, and organises entries into 10-second time windows.
③ Behavioural Detection	At window close, Python calculates features (error rate, requests per IP, path counts, request rate, etc.) and applies deterministic rules to produce a trigger score. When the score crosses a threshold, an investigation window is created — no AI is involved here.
④ Investigation	A local knowledge retriever appends relevant cybersecurity reference text (OWASP, CWE patterns) to the investigation context. The structured investigation object — logs, features, trigger reasons, knowledge — is handed off to Gemini.
⑤ AI Analysis (Gemini)	GeminiClient.analyze_investigation() sends a token-budgeted prompt (hard cap: ~5,500 tokens) and receives a structured SecurityFinding JSON: severity, attack type, confidence score, evidence items, and recommended actions.
⑥ Source-Code Audit	An analyst requests a Quick Scan or Deep Scan audit via POST /audit/request. The AuditEngine crawls the registered application's source directory, scores files by security relevance, builds a token-bounded prompt, and calls Gemini's code-audit path, which returns an array of AuditFinding objects (vulnerability type, CWE, OWASP, file path, line number, explanation, remediation).

- | | |
|-----------------------------|---|
| ⑦ Sandbox Validation | The Advanced AI workflow takes an audit or SOC finding, creates a logical sandbox record pointing at the authorized demo target, crawls its live attack surface, asks Groq to select a controlled validation strategy, then executes it deterministically — HTTP requests with payloads from a fixed allow-list only. Evidence is collected and evaluated to produce a VULNERABLE / MITIGATED / INCONCLUSIVE verdict. |
| ⑧ Security Finding | The full chain — SOC investigation → audit findings → sandbox verdict — is stored in the history store and presented to the analyst dashboard for review and a final fix decision. |

3. AI USAGE

PYTHON (DETERMINISTIC)

- Log ingestion and windowing
- Feature calculation (error rate, IP counts, etc.)
- Trigger score computation and threshold rules
- Source file crawling and security-priority scoring
- Token-budget enforcement on all prompts
- Payload execution during sandbox validation
- Indicator detection (DB error signatures, reflection)
- Evidence redaction (cookies, tokens)

AI (INTERPRETATION)

- Interpreting suspicious log patterns in natural language
- Identifying vulnerability types, CWE, and OWASP categories in source code
- Selecting an appropriate validation strategy for a finding
- Evaluating collected sandbox evidence and explaining the verdict
- Providing remediation guidance
- Answering analyst questions via the Copilot endpoint

Two AI providers are used for distinct roles. **Google Gemini** handles SOC investigation analysis and source-code audits (via `google-generativeai`). **Groq** (OpenAI-compatible API, model `openai/gpt-oss-20b`) handles the Advanced AI planning and evaluation steps and the Copilot chat endpoint. Both are accessed through structured prompts; every response is validated through Pydantic models before use, and any failure degrades to a deterministic fallback rather than crashing the job.

4. SECURITY VALIDATION (ADVANCED AI WORKFLOW)

The Advanced AI pipeline runs a seven-state job (`queued` → `preparing_sandbox` → `crawling` → `analyzing` → `planning_tests` → `validating` → `evaluating` → `completed`) against the Meridian Analytics demo target — the only application in the registry with `advanced_ai_approved=True` and a configured `internal_base_url`.

The sandbox is a logical isolation record: it resolves the target URL from the server-side registry (never from client input), performs a real HTTP reachability probe, and marks the sandbox failed immediately if the target does not respond. No Docker containers or separate processes are spawned.

The only payloads the executor can send are those in a fixed Python allow-list:

- `none` — plain request (no payload)
- `benign` — the string `"sentinel"`

- `sqli_error_probe` — a single quote ('), sufficient to break naive string interpolation and produce a database error without extracting or modifying data
- `xss_reflection_marker` — the alphanumeric string "sentinelxss9f3a"

The SQL injection demonstration is the clearest working scenario: the probe sends a single quote to the Meridian search endpoint; the deterministic evaluator checks for a `sqlite3.operationalerror` or related DB error signature in the response while confirming the benign baseline produced no such error. Only this behavioral difference — not the mere sending of the probe — is used to classify the result as **VULNERABLE**.

VULNERABLE

MITIGATED

INCONCLUSIVE

Groq's role in this phase is purely reasoning: given the finding and the discovered attack surface, it selects which `payload_kind` and which endpoint to test. It does not execute anything. If Groq is unavailable or returns malformed output, a deterministic fallback planner runs instead.

5. HUMAN OVERSIGHT

The system is designed around an explicit human-in-the-loop model. No changes are made to any production or staging system automatically.

- Audit requests are initiated by a human analyst via `POST /audit/request`, not triggered automatically.
- Advanced AI jobs require an explicit human request and are scoped to pre-registered applications only.
- When a sandbox validation returns a **VULNERABLE** verdict, the finding carries a `fix_status: pending` field. The analyst must explicitly mark it `fix_applied` or `fix_rejected` before any remediation action is taken — the system does not apply fixes.
- JWT-based authentication guards all protected endpoints; analyst credentials are loaded from environment variables only.

Sentinel recommends remediation steps (e.g. "use parameterized queries") but never modifies source code, deploys patches, or touches the target application's database.

6. TECHNOLOGY STACK

Python 3.11FastAPI 0.109Uvicorn 0.27Pydantic 2.5httpx 0.26python-dotenv 1.0python-jose (JWT)passlib

Google Gemini API (google-generativeai)Groq API (openai-compatible, openai/gpt-oss-20b)

HTML / CSS / JavaScript (frontend dashboard)SQLite (Meridian demo target)Flask (Meridian demo target)

JSON file store (history)threading.Lock (concurrency)

The backend is a single FastAPI process. All state (investigations, audits, Advanced AI jobs) is held in-memory during a session; the history store is persisted to a local JSON file (`data/history.json`). There is no external database for the Sentinel backend itself.

7. DEVELOPMENT & IBM BOB USAGE

IBM Bob was used throughout development as a hands-on coding assistant. Specific contributions visible in the repository include:

- **Code generation and iteration:** Scaffolding the FastAPI application structure, the log processor windowing logic, the audit engine snippet selector, and the Advanced AI orchestrator state machine.
- **Prompt and token optimization:** Designing the multi-budget token enforcement system (`_enforce_investigation_token_limit`, `_enforce_audit_token_limit`) and tuning the constants (`_INV_HARD_TOKEN_LIMIT`, `_AUDIT_HARD_TOKEN_LIMIT`) to keep prompts within API limits while preserving quality.
- **Debugging:** Diagnosing Groq empty-response edge cases, which produced the detailed `_diag_response_metadata` function and a dedicated test file (`test_groq_empty_response_diagnostics.py`).
- **Test coverage:** Writing the extensive test suite (15+ test files), including regression tests, security boundary tests, pipeline scenario tests, and audit response parsing tests.
- **Security design review:** Catching and enforcing rules such as "Gemini never selects free-form payloads," "client-supplied URLs are rejected," and "internal source paths are never serialized to the frontend."

8. CURRENT LIMITATIONS

- **External API rate limits:** Both Gemini and Groq impose rate limits. The Groq provider includes defensive empty-response handling and a deterministic fallback, but sustained high load will exhaust the free tier.
- **In-memory state:** Investigations and audit results are lost on process restart (only the history JSON file survives). There is no database backing the live pipeline state.
- **Single authorized Advanced AI target:** The sandbox validation path currently works only against the Meridian Analytics demo application. Adding a new target requires editing the server-side registry and running the demo app separately.
- **Log ingestion is push-only:** Logs are sent to `POST /logs` by a caller (e.g. `generate_logs.py`); the system does not pull logs from real SIEM platforms or log aggregators.
- **No RAG knowledge base:** The local knowledge retriever is a keyword-matched flat file rather than a vector database. CVE/CWE coverage is limited to what is hard-coded in the knowledge files.
- **Prototype-scale frontend:** The dashboard is a single HTML/JS/CSS page; it has no build pipeline, no persistent session management beyond a JWT stored in memory, and no real-time WebSocket updates.
- **AI outputs are probabilistic:** Gemini and Groq findings are not guaranteed to be correct. Every AI-produced finding should be reviewed by an analyst before action is taken.

9. DEMO SCENARIO

The strongest end-to-end demonstration runs against the Meridian Analytics Flask application:

1. Start the Meridian app on `localhost:5001` and the Sentinel backend on `localhost:8000`.
2. Run `generate_logs.py` to push a burst of simulated HTTP logs — including repeated failed logins and SQL-injection-pattern requests — to `POST /logs`.

3. The LogProcessor triggers an investigation within 10 seconds. Gemini produces a SecurityFinding identifying the suspicious behavior as a probable SQL injection attempt, with confidence and recommended actions.
4. From the dashboard, the analyst launches a Quick Scan audit. The AuditEngine crawls Meridian's Python source, prioritises authentication and database routes, and Gemini returns an AuditFinding pointing to the raw string interpolation in the search endpoint, tagged CWE-89 (SQL Injection).
5. The analyst submits the audit finding to the Advanced AI workflow. The orchestrator crawls the live Meridian attack surface, Groq plans an `sqli_error_probe` test on `/api/search?q=`, the executor sends a single quote, the response contains an `sqlite3.operationalerror`, the benign baseline does not, and the job completes with verdict **VULNERABLE** and a concrete remediation recommendation.
6. The analyst reviews the full evidence trail and marks the finding for remediation.

10. FINAL IMPACT

Sentinel AI is useful to a SOC analyst because it compresses a multi-step security workflow — log triage, pattern recognition, code review, and vulnerability confirmation — into a single, auditable pipeline. The analyst does not have to manually correlate 400 HTTP log lines, grep through a codebase, or run custom exploit scripts to confirm a finding. The system surfaces where to look and why, produces an evidence trail that explains its conclusions, and then holds for human approval before anything further happens.

The design deliberately keeps AI in the interpretation layer and Python in the execution layer. This means a rate-limited or unavailable AI provider degrades gracefully to deterministic results rather than producing silent failures, and it means no AI model can spontaneously issue HTTP requests, modify code, or escalate privileges. That boundary is enforced in code, not just in policy.

For a five-day prototype, the result is a working pipeline that goes all the way from a raw log stream to a sandbox-confirmed vulnerability verdict — something a real SOC team would need multiple specialized tools to replicate.

Made with IBM Bob